

INFORME TÉCNICO

Investigación y Consumo de API Externa EmailJS + Proyecto AyL compresores y partes

Instructor: Ing. Edwin Marín | Abril 2026

GRUPO: A&L COMPRESORES Y PARTES

1. API SELECCIONADA: EmailJS

1.1 Justificación de la elección

EmailJS permite enviar correos electrónicos directamente desde el cliente (frontend) o mediante un servidor sencillo sin necesidad de configurar un backend complejo de correo. Se integra fácilmente con servicios como Gmail, Outlook y otros.

Necesidad en AyL compresores y partes: Notificar a la empresa sobre las cotizaciones que por medio de un formulario básico el cliente llena. El formulario permite una vía de comunicación directa con los deseos de los clientes para que se esa forma

Necesidad en tu proyecto	Solución con api externa
Permitir que los clientes envíen solicitudes de contacto o cotización desde la página web.	EmailJS permite enviar correos directamente desde el frontend usando formularios HTML
Evitar implementar un backend solo para envío de correos.	Se integra fácilmente con React mediante <code>emailjs.sendForm</code> .
Tener una comunicación rápida entre cliente y empresa.	No requiere servidor propio para el envío de emails.

El flujo : Cliente llena formulario → front llama EmailJS→ empresa recibe el correo generado con la información diligenciada por el cliente

1.2 Problema que resuelve la API

2. INVESTIGACIÓN TÉCNICA DE LA API

2.1 Documentación oficial

URL de documentación: <https://www.emailjs.com/docs/>

2.2 Métodos HTTP y endpoints clave

Método	Endpoint	Uso en	Respuesta clave
POST	<code>/api/v1.0/email/send</code>	Envío de correo mediante JSON	200 OK
POST	<code>/api/v1.0/email/send-form</code>	Envío desde un formulario HTML	200 OK

2.3 Autenticación

EmailJS utiliza autenticación mediante claves públicas:

- **Public Key (User ID)** → usada en el frontend
- **Service ID** → identifica el servicio de correo
- **Template ID** → define el formato del correo

```
emailjs.sendForm(  
  'service_ecsbd4v',  
  'template_3ctxgvs',  
  form.current,  
  'ZDygnYA16n4gwAS4p'  
)
```

2.4 Límites de uso y políticas

Aspecto	Detalle
Plan gratuito	200 emails/mes, 1 servicio de email, 2 plantillas de email.
Plan Personal	Desde USD 9/mes hasta 2.000 emails/mes.

3. IMPLEMENTACIÓN TÉCNICA

3.1 Estructura del proyecto de demostración ejemplo

```
ayl-compresores/  
├── public/  
│   └── images/  
│       └── Nosotros/somos.png  
├── src/  
│   ├── components/  
│   │   ├── Home/  
│   │   │   ├── Navbar.jsx  
│   │   │   ├── Footer.jsx  
│   │   │   └── CTA.jsx  
│   │   ├── CartPanel.jsx  
│   │   └── LoginModal.jsx  
│   ├── pages/  
│   │   └── ContactoPage.jsx    # Formulario con EmailJS  
│   └── App.jsx  
├── package.json  
└── README.md
```

3.2 Consumo funcional — Notificación de cambio de estado (POST)

Ejemplo : Este endpoint POST envía un email al cliente cada vez que el admin actualiza el estado de su orden en Texticode:

```
const enviarEmail = (e) => {  
  e.preventDefault();  
  
  Swal.fire({  
    title: 'Enviando mensaje...',  
    text: 'Por favor, espera un momento.',  
    allowOutsideClick: false,  
    didOpen: () => {  
      Swal.showLoading();  
    }  
  });  
  
  emailjs.sendForm(  
    'service_ecsbd4v',  
    'template_3ctxgvs',  
    form.current,  
    'ZDygnYA16n4gwAS4p'  
  )  
  .then((result) => {  
  
    Swal.fire({  
      icon: 'success',  
      title: '¡Enviado!',  
      text: 'Mensaje enviado con éxito a A&L Compresores',  
    });  
  
    e.target.reset();  
  }, (error) => {
```

```

    Swal.fire({
      icon: 'error',
      title: 'Error al enviar',
      text: 'No se pudo enviar el mensaje',
    });
  });
};

try {
  await sgMail.send(msg);
  res.json({ success: true, mensaje: 'Email enviado al cliente.' });
} catch (error) {
  res.status(500).json({ success: false, error: error.message });
}
});

```

3.3 Consumo GET — Consultar estadísticas de envíos

En este proyecto no se implementan endpoints GET, ya que EmailJS está diseñado principalmente para el envío de correos (POST).

3.4 Envío de comprobante con adjunto PDF

```

// Envío del comprobante adjunto en PDF
router.post('/enviar-comprobante', async (req, res) => {
  const { ordenId, clienteEmail, pdfBase64 } = req.body;
  const msg = {
    to: clienteEmail,
    from: process.env.SENDGRID_FROM_EMAIL,
    subject: `Texticode - Comprobante de entrega Orden #${ordenId}`,
    text: `Adjunto encontrarás el comprobante de entrega de tu orden #${ordenId}.`,
    attachments: [{
      content: pdfBase64, // PDF en base64
      filename: `comprobante_${ordenId}.pdf`,
      type: 'application/pdf',
      disposition: 'attachment'
    }]
  };
};

try {
  await sgMail.send(msg);
  res.json({ success: true });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

```

4. SEGURIDAD Y BUENAS PRÁCTICAS

4.1 Protección de credenciales

No subir claves sensibles a repositorios públicos.

Usar variables de entorno (`.env`) en producción.

Evitar exponer claves privadas (EmailJS usa public key, pero igual se debe proteger).

```
REACT_APP_EMAILJS_PUBLIC_KEY=xxxxx
REACT_APP_EMAILJS_SERVICE_ID=xxxxx
REACT_APP_EMAILJS_TEMPLATE_ID=xxxxx
```

4.2 Riesgos de seguridad identificados

Riesgo	Mitigación aplicada
Uso indebido del servicio	Configuración de límites en EmailJS
Exposición de la public key	Uso limitado y restricciones desde EmailJS

5. PREGUNTAS DE INVESTIGACIÓN

P1. ¿Qué es EmailJS y para qué sirve?

Es un servicio que permite enviar correos electrónicos desde el frontend sin necesidad de backend.

P2. Métodos HTTP y datos que retorna

- Método: **POST**
- Endpoint: **/api/v1.0/email/send-form**
- Respuesta:
 - **200 OK** → envío exitoso
 - **Error** → fallo en el envío

P3. ¿Cómo se autentica?

- Mediante Public Key
- Service ID
- Template ID

P4. Límites gratuitos y políticas de uso

Plan gratis: ~200 correos/mes

Limitaciones:

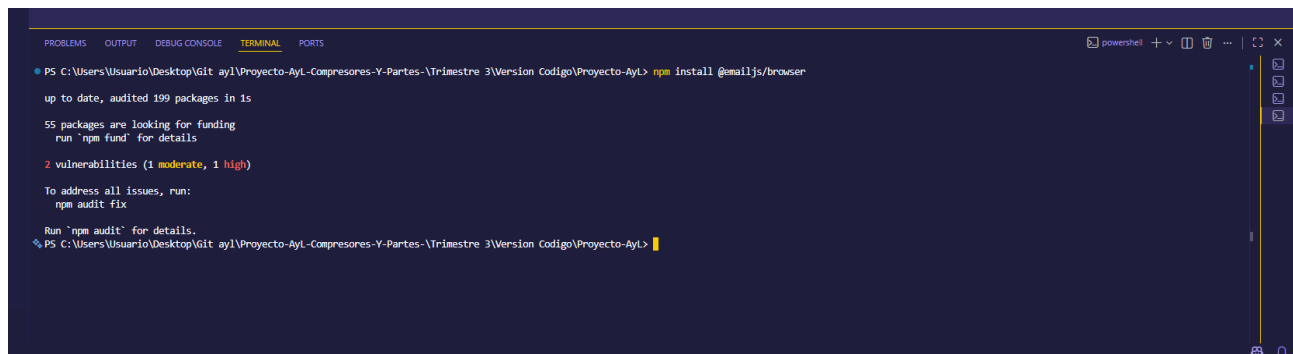
- Número de plantillas
- Servicios de correo

P5. Riesgos de seguridad

Exposición de claves
Spam
Uso malintencionado del formulario

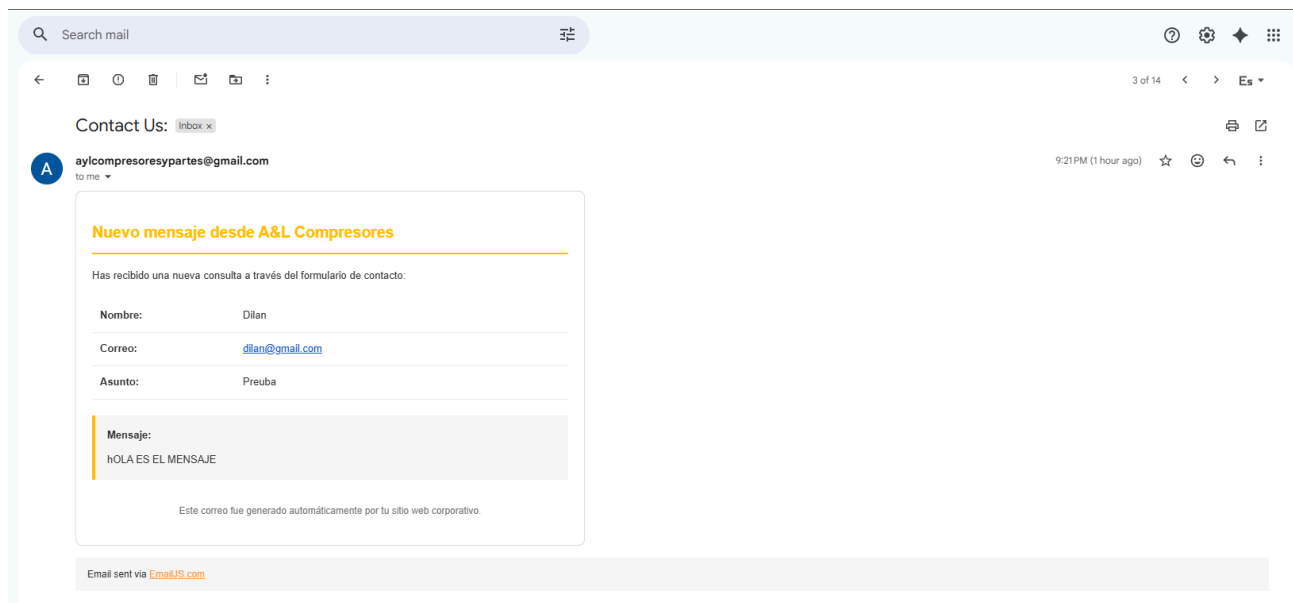
6. EVIDENCIAS DE PRUEBA

6.1 Servidor corriendo



```
PS C:\Users\Usuario\Desktop\Git_ayl\Proyecto-AyL-Compresores-Y-Partes-Trimestre 3\VersionCodigo\Proyecto-AyL> npm install @emailjs/browser
up to date, audited 199 packages in 1s
55 packages are looking for funding
run 'npm fund' for details
2 vulnerabilities (1 moderate, 1 high)
To address all issues, run:
  npm audit fix
Run 'npm audit' for details.
PS C:\Users\Usuario\Desktop\Git_ayl\Proyecto-AyL-Compresores-Y-Partes-Trimestre 3\VersionCodigo\Proyecto-AyL>
```

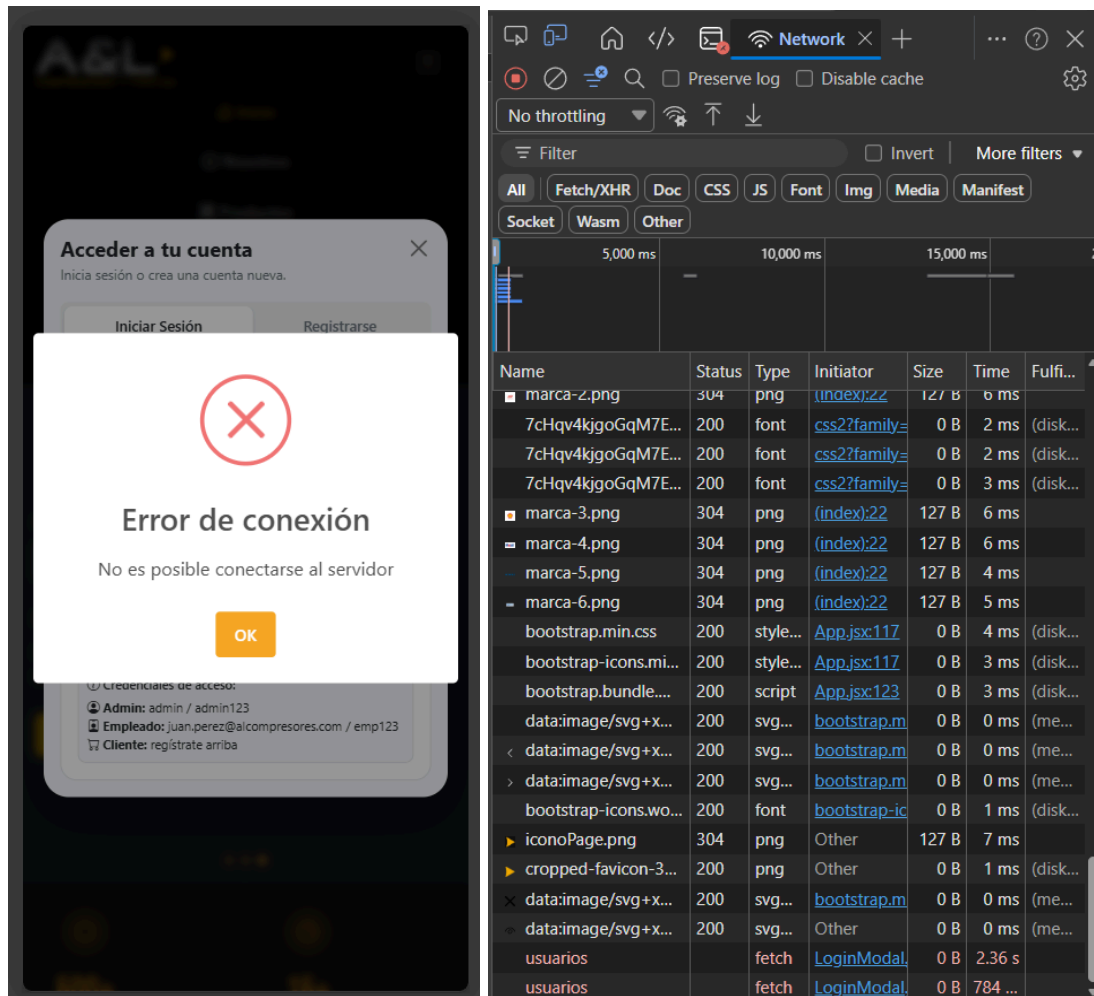
6.2 Prueba exitosa — Enviar correo



6.3 Prueba exitosa — Consultar estadísticas (GET)

The image is a composite of two screenshots. The left screenshot shows a mobile application interface with a dark theme. At the top, there's a logo 'A&L' with the text 'COMPRESORES Y PINTAS' below it. A hamburger menu icon is on the right. The main menu has items: 'Inicio', 'Nosotros', 'Productos', and 'Contactos'. Below these is a shopping cart icon and a button labeled 'Iniciar sesión'. A large white modal is centered on the screen, featuring a green checkmark icon and the text 'Inicio exitoso'. Below the modal, there's a line of text in Spanish: 'Más de 500 referencias en stock con despacho en 48 horas a toda Colombia.' At the bottom, there are two buttons: 'Ver Productos' and 'Contáctanos'. The right screenshot shows a web browser interface. At the top, there's a search bar with the text 'No throttling'. Below it, there's a 'Filter' button and a 'More filters' dropdown. A row of buttons includes 'All', 'Fetch/XHR', 'Doc', 'CSS', 'JS', 'Font', 'Img', 'Media', and 'Manifest'. Below these are buttons for 'Socket', 'Wasm', and 'Other'. A timeline view shows a sequence of events with markers at 5,000 ms, 10,000 ms, and 15,000 ms. Below the timeline is a table with columns: 'Name', 'Status', 'Type', 'Initiator', 'Size', 'Time', and 'Fulfi...'. The table lists various files and their properties, including 'products.jsx', 'bootstrap.min.css', 'bootstrap.bundle.js', and 'bootstrap-icons.min.css'.

6.4 Prueba de error — Email inválido



7. INTEGRACIÓN PROPUESTA

7.1 Flujo de notificación completo

#	Actor	Acción
1	Cliente	Llena el formulario de contacto
2	Frontend (React)	Ejecuta <code>email.send Form()</code>

3	EmailJS	Procesa la solicitud
4	Servicio de correo	Envía el email
5	Empresa	Recibe el mensaje del cliente

7.2 Fragmento Vue propuesto (xxxxx.vue)

Nota:

El envío de correos se realiza desde un componente React (**ContactoPage**) utilizando la librería **emailjs**. Se usa **useRef** para capturar el formulario y enviarlo directamente.

Links:

comparación de apis externas: ejemplo

https://postmarkapp.com/compare/sendgrid-alternative?utm_source=google&utm_medium=cpc&utm_campaign=Postmark_Google_Search_Non_Competitive_Sendgrid&utm_adgroup=API&utm_term=sendgrid%20email%20api&gad_source=1&gad_campaignid=22313661785&gbraid=0AAAAAqOJNabl6fUBgm5RoBXA330fdOm5X&gclid=Cj0KCQjw2MbPBhCSARIsAP3jP9x7tyGwWYzPUI7LirxEBH1oAwGyCH1b7k3nDZYXtKdX0DhHOMfhtEoaArxgEALw_wcB#sending

de apoyo | Instructor: Ing. Edwin Marín